

El árbol de decisión por dentro: argumentos, ejecución y métricas

Laboratorio de Python paso a paso · Machine Learning · Corte 3 · UTB 2026-II

Jorge Luis Villalba Acevedo

2026-09-22

Tabla de contenidos

1	Las tres fases	2
2	Fase 1 · Los argumentos	2
3	Fase 2 · Entrenar: cómo elige la primera pregunta	3
3.1	La matemática	3
3.2	A mano: la búsqueda exhaustiva	4
3.3	Qué guarda el árbol: la estructura <code>tree_</code>	4
4	Leer el árbol: dibujo y reglas	5
5	Fase 3 · Usar: cómo predice	6
6	El efecto de cada argumento	6
7	Árbol contra bosque	7
8	Las métricas: cómo se llaman bien	8

i Para qué sirve este laboratorio

Abrimos `DecisionTreeClassifier`, **un solo árbol**: sus argumentos, cómo elige cada pregunta (lo reproduciremos **a mano** buscando el mejor corte), qué guarda por dentro, cómo se lee como reglas y cómo se llaman bien las métricas. Sobre la misma `cartera_curso.csv`.

Al final lo comparamos con un **Random Forest**, que es un conjunto de muchos árboles como éste, para que quede clara la diferencia.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import Markdown, display

from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
```

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix)

plt.rcParams.update({"figure.dpi": 140, "font.size": 9,
                    "axes.spines.top": False, "axes.spines.right": False})
ROJO, VERDE, AZUL, NARANJA, MORADO, GRIS = ("D81B60", "#009E73", "#0072B2",
                                             "#E69F00", "#8E44AD", "#999999")

def tabla(df, decimales=3):
    d = df.copy()
    for c in d.columns:
        if pd.api.types.is_float_dtype(d[c]):
            d[c] = d[c].map(lambda v: f"{v:.{decimales}f}")
    cab = "| " + " | ".join(str(c) for c in d.columns) + " |"
    sep = "|" + "|" * len(d.columns) + "|"
    fil = ["| " + " | ".join(str(v) for v in r) + " |" for r in d.values]
    return Markdown("\n".join([cab, sep] + fil))

def texto(md):
    display(Markdown(md))

```

```

datos = pd.read_csv("../cartera_curso.csv", sep=";", decimal=",")
V, OBJ = ["edad", "ingreso"], "tomo_credito"

d = datos.sort_values("id_cliente").reset_index(drop=True)
corte = int(np.floor(0.75 * len(d)))
X_train, y_train = d.loc[:corte-1, V].values, d.loc[:corte-1, OBJ].values
X_test, y_test = d.loc[corte:, V].values, d.loc[corte:, OBJ].values

```

1. Las tres fases

Fase	Código	Qué pasa en el árbol
1. Construir	<code>DecisionTreeClassifier(max_depth=2, ...)</code>	Se fija qué tanto puede crecer el árbol.
2. Entrenar	<code>.fit(X_train, y_train)</code>	En cada nodo prueba todos los cortes posibles y se queda con el que más reduce el Gini. La estructura queda en <code>tree_</code> .
3. Usar	<code>.predict(X)</code>	Cada cliente baja por las preguntas hasta una hoja y recibe la clase mayoritaria de esa hoja.

2. Fase 1 · Los argumentos

```
params = DecisionTreeClassifier().get_params()
```

Tabla 2: Argumentos de DecisionTreeClassifier. La columna «de fábrica» sale de get_params().

argumento	de fábrica	qué controla	cuándo tocarlo
criterion	gini	cómo mide la impureza	'gini' casi siempre; 'entropy' da árboles muy parecidos
splitter	best	cómo elige el umbral de cada corte	'best' prueba todos; 'random' sortea (útil en conjuntos de árboles)
max_depth	None	cuántas preguntas encadenadas	None = crece hasta hojas puras (memoriza)
min_samples_split	2	mínimo de clientes para partir un nodo	subirlo evita cortes sobre grupos diminutos
min_samples_leaf	1	mínimo de clientes en cada hoja	subirlo suaviza la frontera
max_leaf_nodes	None	número máximo de hojas	otra forma de limitar el tamaño del árbol
min_impurity_decrease	0.0	ganancia mínima para aceptar un corte	evita cortes que casi no mejoran nada
ccp_alpha	0.0	poda por costo-complejidad	mayor que 0 = poda las ramas que no pagan su costo
class_weight	None	peso de cada clase	'balanced' con clases muy desbalanceadas
random_state	None	semilla	fixarla: con empates entre cortes, decide cuál se elige

```
arbol = DecisionTreeClassifier(  
    criterion="gini",      # impureza de Gini  
    splitter="best",      # probar todos los cortes y quedarse con el mejor  
    max_depth=2,          # a lo sumo dos preguntas encadenadas  
    min_samples_leaf=5,   # ninguna hoja con menos de 5 clientes  
    random_state=0)       # reproducible
```

Árbol contra bosque en los argumentos. El árbol **no tiene** `n_estimators` ni `bootstrap`: esos son del bosque, porque hablan de cuántos árboles hay y de cómo se muestrea para cada uno. Y en el árbol `max_features` viene en `None` (mira todas las variables), mientras en el bosque viene en `'sqrt'` (cada corte mira solo algunas, para que los árboles salgan distintos).

3. Fase 2 · Entrenar: cómo elige la primera pregunta

3.1. La matemática

La impureza de un grupo con proporción p de la clase 1 es $G = 2p(1 - p)$. El árbol prueba cada variable y cada umbral posible, y se queda con el corte que minimiza el Gini ponderado:

$$G_{\text{corte}} = \frac{n_{\text{izq}}}{n} G_{\text{izq}} + \frac{n_{\text{der}}}{n} G_{\text{der}} \quad \text{ganancia} = G_{\text{raíz}} - G_{\text{corte}}$$

3.2. A mano: la búsqueda exhaustiva

```
def gini(y):
    if len(y) == 0:
        return 0.0
    p = y.mean()
    return 2 * p * (1 - p)

g_raiz = gini(y_train)
mejor = {"gp": np.inf}
for j, nombre in enumerate(V):
    valores = np.unique(X_train[:, j])
    for u in (valores[:-1] + valores[1:]) / 2:          # puntos medios, como sklearn
        izq = X_train[:, j] <= u
        if izq.sum() < 5 or (~izq).sum() < 5:          # respeta min_samples_leaf=5
            continue
        gp = (izq.sum() * gini(y_train[izq]) + (~izq).sum() * gini(y_train[~izq])) / len(y_train)
        if gp < mejor["gp"]:
            mejor = {"variable": nombre, "umbral": u, "gp": gp}

arbol.fit(X_train, y_train)
t = arbol.tree_
```

El Gini de la raíz es **0.4982**. La búsqueda a mano probó todos los umbrales de las dos variables y el mejor corte es **¿ingreso 6.55?**, con Gini ponderado **0.3803** y ganancia **0.1179**.

`.fit()` eligió **¿ingreso 6.55?**, con Gini ponderado **0.3803**: la misma pregunta. Eso es todo lo que hace un árbol al entrenar: repetir esta búsqueda en cada nodo nuevo.

Por qué el umbral es un punto medio (por ejemplo 6.55 y no 6.5 o 6.6): el árbol corta entre dos valores observados consecutivos, así que cualquier número entre ellos separa igual; scikit-learn usa el punto medio.

3.3. Qué guarda el árbol: la estructura `tree_`

Tabla 3: La estructura interna del árbol: una fila por nodo.

nodo	tipo	pregunta	clientes	proporción clase 1	Gini
0	pregunta	ingreso ≤ 6.55	300	0.530	0.498
1	pregunta	edad ≤ 45.50	194	0.351	0.455
2	hoja	—	136	0.125	0.219
3	hoja	—	58	0.879	0.212
4	pregunta	edad ≤ 53.00	106	0.858	0.243
5	hoja	—	89	0.831	0.280
6	hoja	—	17	1.000	0.000

Tabla 4: Atributos principales después de .fit().

atributo	qué contiene	valor
tree_	toda la estructura (la tabla anterior)	7 nodos
get_depth()	profundidad alcanzada	2
get_n_leaves()	número de hojas	4
feature_importances_	reducción total de Gini por variable	edad: 0.571, ingreso: 0.429
classes_	orden de las columnas de predict_proba	[0, 1]

4. Leer el árbol: dibujo y reglas

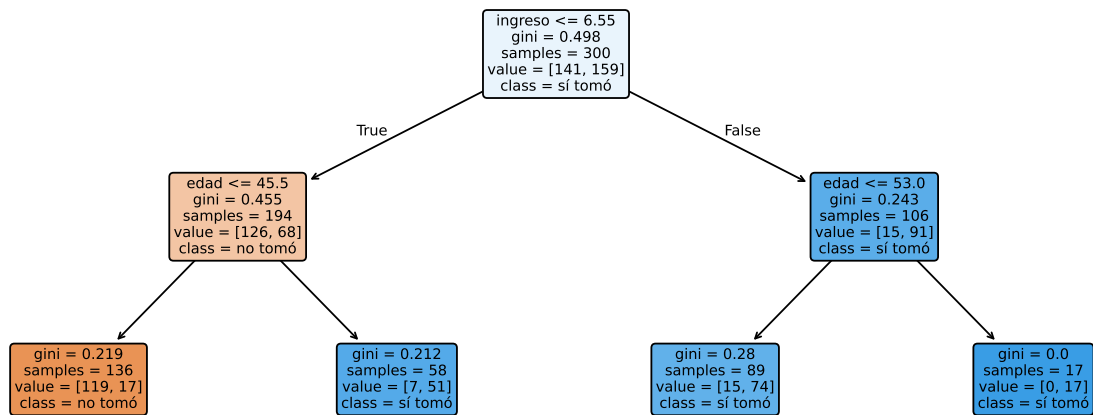


Figura 1: El árbol entrenado. En cada caja: la pregunta, el Gini, los clientes y cómo se reparten.

La misma información, como texto, con `export_text`, que es lo que se pega en un informe o se le explica a un comité:

```
reglas = export_text(arbol, feature_names=V, decimals=2)
cerca = "`" * 3 # delimitador de bloque de código en Markdown
texto(cerca + "\n" + reglas + cerca)
```

```
|--- ingreso <= 6.55
|   |--- edad <= 45.50
|   |   |--- class: 0
|   |   |--- edad > 45.50
|   |   |--- class: 1
|--- ingreso > 6.55
|   |--- edad <= 53.00
|   |   |--- class: 1
|   |   |--- edad > 53.00
|   |   |--- class: 1
```

Esta es la ventaja que el bosque no tiene. Un árbol se puede leer entero como un puñado de reglas «si... entonces». Un bosque de 200 árboles no: gana precisión, pero pierde esta explicación.

5. Fase 3 · Usar: cómo predice

La probabilidad que da un árbol es **la proporción de la clase 1 en la hoja** donde cae el cliente.

```
hojas = arbol.apply(X_test) # en qué hoja cae cada cliente
p_mano = t.value[hojas, 0, 1] / t.value[hojas, 0].sum(axis=1)
p_sklearn = arbol.predict_proba(X_test)[: , 1]
```

El primer cliente de la prueba (edad 48, ingreso 7.2) cae en la **hoja 5**, donde la proporción de clientes que tomaron el crédito es **0.8315**. Ésa es su probabilidad, y `predict_proba` devuelve exactamente eso para todos los clientes (**sí**).

Consecuencia: un árbol de 4 hojas solo puede dar **4 probabilidades distintas**. Todos los clientes de una misma hoja reciben la misma.

6. El efecto de cada argumento

```
def probar(**cambio):
    m = DecisionTreeClassifier(random_state=0, **cambio).fit(X_train, y_train)
    return (m.get_n_leaves(), m.score(X_train, y_train), m.score(X_test, y_test))

experimentos = []
for nombre, valores in [("max_depth", [1, 2, None]),
                        ("min_samples_leaf", [1, 5, 30]),
                        ("ccp_alpha", [0.0, 0.005, 0.02])]:
```

```

for v in valores:
    hojas_n, tr, te = probar(**{nombre: v})
    experimentos.append({"argumento": nombre, "valor": str(v), "hojas": hojas_n,
                        "entrenamiento": tr, "prueba": te, "brecha": tr - te})
experimentos = pd.DataFrame(experimentos)

```

Tabla 5: Efecto de mover un argumento a la vez; los demás en su valor de fábrica.

argumento	valor	hojas	entrenamiento	prueba	brecha
max_depth	1	2	0.723	0.700	0.023
max_depth	2	4	0.870	0.900	-0.030
max_depth	None	75	0.980	0.770	0.210
min_samples_leaf	1	75	0.980	0.770	0.210
min_samples_leaf	5	33	0.877	0.870	0.007
min_samples_leaf	30	7	0.870	0.900	-0.030
ccp_alpha	0.0	75	0.980	0.770	0.210
ccp_alpha	0.005	4	0.873	0.890	-0.017
ccp_alpha	0.02	3	0.870	0.900	-0.030

Con los valores de fábrica (`max_depth=None`) el árbol crece hasta **75 hojas**, llega a **0.980** en entrenamiento y la brecha es **+0.210**: memorizó. Con `max_depth=2` bastan **4 hojas** y la prueba es **0.900**.

`ccp_alpha` hace lo mismo por otro camino: **poda después de crecer**, cortando las ramas que no pagan su costo. Con `ccp_alpha=0.02` el árbol queda en **3 hojas**.

7. Árbol contra bosque

```

filas = []
for s in range(20):
    A, B, a, b = train_test_split(datos[V].values, datos[OBJ].values,
                                  test_size=0.25, random_state=s)
    filas.append({
        "árbol": DecisionTreeClassifier(random_state=0).fit(A, a).score(B, b),
        "bosque": RandomForestClassifier(n_estimators=200, random_state=0,
                                         n_jobs=-1).fit(A, a).score(B, b)})
comp = pd.DataFrame(filas)

```

Tabla 6: Un árbol y un bosque, ambos con sus argumentos de fábrica, sobre 20 particiones aleatorias.

modelo	exactitud media	desviación	se puede leer como reglas
Árbol de decisión (uno solo)	0.765	0.037	sí
Random Forest (200 árboles)	0.831	0.037	no

Con los dos modelos en sus valores de fábrica, el árbol solo promedia **0.765** de exactitud con desviación **0.037**, y el bosque **0.831** con desviación **0.037**. Promediar muchos árboles, cada uno entrenado con una muestra distinta, **compensa la tendencia de cada árbol a memorizar**. El precio es que el bosque ya no se puede leer como reglas.

8. Las métricas: cómo se llaman bien

```
clase = arbol.predict(X_test)
exactitud = accuracy_score(y_true=y_test, y_pred=clase)
precision = precision_score(y_true=y_test, y_pred=clase, pos_label=1, zero_division=0)
recall    = recall_score(y_true=y_test, y_pred=clase, pos_label=1)
f1        = f1_score(y_true=y_test, y_pred=clase, pos_label=1)
VN, FP, FN, VP = confusion_matrix(y_true=y_test, y_pred=clase, labels=[0, 1]).ravel()
```

Tabla 7: Cada métrica del árbol, su llamada y su valor.

métrica	llamada	fórmula	valor
exactitud	<code>accuracy_score(y_true, y_pred)</code>	$(VP+VN)/n$	0.900
precisión	<code>precision_score(y_true, y_pred, pos_label=1)</code>	$VP/(VP+FP)$	0.898
recall	<code>recall_score(y_true, y_pred, pos_label=1)</code>	$VP/(VP+FN)$	0.930
F1	<code>f1_score(y_true, y_pred, pos_label=1)</code>	$2PR/(P+R)$	0.914

Matriz de confusión con `confusion_matrix(y_true, y_pred, labels=[0, 1])`:

	pred. 0	pred. 1	total
real 0	37	6	43
real 1	4	53	57
total	41	59	100

i Lo que usted entrega

1. Renderice a **PDF** con el código visible.
2. Responda:
 - a. Con la tabla 3: ¿cuántos clientes caen en cada hoja y qué probabilidad recibe un cliente de cada una?
 - b. Cambie `min_samples_leaf=5` por 1 en la búsqueda a mano y en el árbol: ¿sigue coincidiendo la primera pregunta?
 - c. Copie las reglas de `export_text` y explíquelas en palabras del negocio, como si se las presentara al comité de crédito.
 - d. Con la tabla 6: ¿cuándo preferiría un árbol y cuándo un bosque?